Nama : Arief Purnama Muharram
NIM : 23521013

SQL → Stored in columns and tables

DML → Data Manipulation Language → insert, update, delete, select

DDL → Data Definition Language
- → Primary key
- → Composite key
- → Foreign key

ACID → SQL

A : Atomic → transaction complete or not

C : Consistent → transaction is consistent
↳ defined in terms of constraint

I : Isolated → the result of any changes made by during a transaction are not visible until the transaction has committed

D : Durable → The result of a committed transaction survive failures

BASE → in NoSQL

Basically Available → Nodes can go up and down but the system shouldn't affected

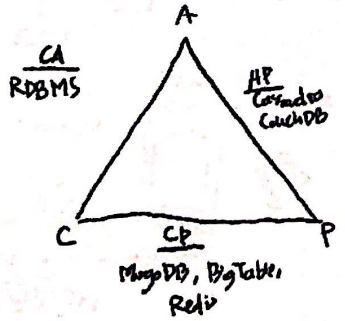
Soft state (scalable) → The state of the system and data changes over time

Eventually consistent → Given enough time, data will be consistent across the distributed system ("pada akhirnya sinkron juga")

Brewer's CAP Theorem

A distributed system can only support two of following characteristics:

Consistency ⇒ All nodes see the same data at the same time
Availability → Node failures doesn't make the rest to be failed
Partition tolerance ⇒ The system continues to operate despite arbitrary message loss



NoSQL Database Types

Key-Value Store
Column Store : Hadoop, Cassandra
Document Store
Graph model

Key-values

Pros : Fast, Scalable, Simple, easy to distribute horizontally

Cons : Many data cannot be modeled

Key-Value DB

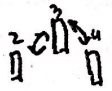
Key ≠ reference

key convention:
<table> : <primary-key> : <attributes>

Namespace → Collection of ~~key pairs~~ key-value pairs
↳ define an additional profile for the keys.

Cluster
↳ Sets to connected computers that coordinate their operations
↳ key value → loosely coupled
↳ type → Masterless (ex Riak)
→ Have master (ex Redis)

Ring
↳ Circular pattern in which each server or instance of key-value database software running on a server is linked to two adjacent servers or instances.



Replication
↳ process of saving multiple copies of data in a cluster

Collision
↳ occurs when two distinct input to a hash function produce the same output.
↳ Compression, hashing, etc.

Design Pattern for Key-Value Database

Time-to-Live (TTL) keys
Expanding tables
Aggregates
Atomic aggregates
Enumerable keys
Indexes

Document ⇒ A set of ordered key-value pairs

Collection → A group of documents

Embedded document → enables document database users to store related data in a single document.

Polymorphic → many forms

Type of partitions : # Vertical partitioning
Horizontal partitioning

Vertical → column splitting

Horizontal → row splitting

split by shard key

Tuning : 1. Index tuning
2. Schema tuning
3. Query tuning
4. Transaction tuning

Index methods : # Ordered index
B^+-tree index
Hash index
Bitmap index

>> create index <index-name> on <rel-name> (<attribute-list>)

>> create index b-index on branch(branch-name)

>> drop index <index-name>

Schema tuning → Horizontal decomposition
→ Vertical composition

Splitting
Horizontally
Vertically
⇓
Column-based

Denormalization
(#) Adding redundant columns
(#) Adding derived attributes
Collapsing tables
Duplicate tables
→ reduction lookup redundancy
→ [illegible] redundancy [illegible] derived attribute

Ensure data integrity :
Trigger
Application logic
Batch reconciliation

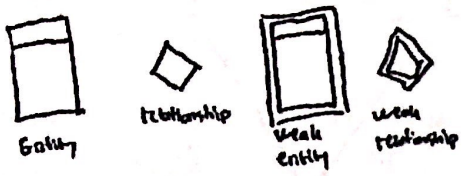
Materialized View → Simpan tabel

>> CREATE MATERIALIZED VIEW <name> AS
>> SELECT ...

Normalisasi bertujuan untuk :
menghindari insertion anomaly
menghindari deletion anomaly
menghindari modification anomaly

Transitive dependency → when some non-key attribute determines some other attribute

Nama : Arief Rusdana Maharam
Nim : 23521013

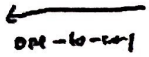


Relationship Set s/ Attr

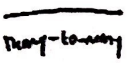
Relationships

one-to-one

many-to-one



one-to-many



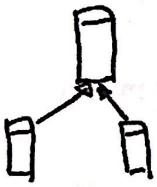
many-to-many

Total participation

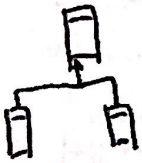
Many-to-one
One-to-many } lebar ke sisi many

kalau many yang punya primary key

kalau one-to-one, pilih salah satu saja!



Overlapping
(bisa juga lebih)



Disjoint
(cukup satu aja)

```
>> CREATE TABLE <name> (
    <attribute> <type> PRIMARY KEY
    FOREIGN KEY ON (<attr>) REFERENCES <table>(<attr>)
    )
```

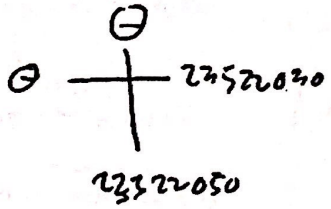
Data Type

# CHAR (size)	# INT (size)	# DATETIME
# VARCHAR (size)	# BIGINT (size)	# TIMESTAMP
# BINARY (size)	# FLOAT (size, d)	# TIME
# TEXT (size)	# DOUBLE (size, d)	# YEAR
# BOOLEAN		

```
>> DROP TABLE <name>
>> TRUNCATE TABLE <name>
>> ALTER TABLE <name> ADD <col> <type>
>> INSERT INTO ... (...) VALUES (...)
>> UPDATE <name> SET <attribute> = "..."
>> PRIMARY KEY(..., ..., ...)

>> SELECT EXTRACT(YEAR, <date>) AS <alias>
>> DELETE FROM <table-name> WHERE <condition>
>> AVG(...)
>> COUNT(...)
>> CREATE TRIGGER <name> {BEFORE|AFTER}
    {INSERT|UPDATE|DELETE} ON <table> FOR
    EACH ROW BEGIN ... END$$ DELIMITER ;
```

(4)



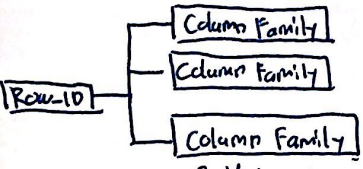
Column-based

A row in a column family database is organized a set of column families

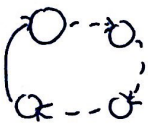
Column families consists of related columns.

A data value is indexed by a row, a column name, and a timestamp.

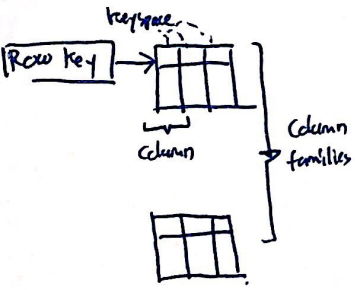
Tx in column must be in a row
↳ Jika multirow, pecah menjadi transaksi setiap row.



Colum. Arch. → Multiple node type
→ peer-to-peer type
↳ Casandra



Components:
- Keyspace
- Row key
- Column
- Column families



Design - column based

Denormalize instead of join

Make use of valueless cd.

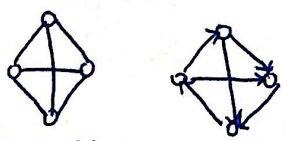
Model an entity with a single row

Use ~~both~~ both column names and column values to store data

Avoid hotspotting in row keys.

Graph-based

Each vertex and edge can have properties.



Properties of graph

Isomorphism
Order (→ number of vertices) and Size (→ number of edges)
Degree → number of edges linked to a vertex
Closeness
Betweenness → How much of a bottleneck a given vertex is

Type of graph

Undirected and directed graph.
Flow networks
Bipartite graphs
Multigraphs
Weighted graphs

Considerations in selecting DBMS:

~~Ex~~ Expected queries
The volume of reads and writes
Tolerance for inconsistent data in replicas
Availability and disaster recovery req.
Latency req.

Transaction

Transaction management → System that allows ~~conf~~ conflict-free simultaneous work by multiple users.

Degree of ~~Cons~~ Consistency

>> SET TRANSACTION ISOLATION LEVEL <isolation level>

options:
- READ UNCOMMITTED : No consistency enforcement
- READ COMMITTED : Only applied changes can be read by other transactions
- ~~READ~~ REPEATABLE READ : Read queries give the same result repeatedly.
- SERIALIZABLE : Full serializable ACID consistency enforced.

~~READ~~ Read Phenomena

- Dirty reads : Read not committed tx is allowed
- Non-repeatable reads : Read only committed tx
- Phantom reads : Repeatable read.

Serializability
- → Pessimistic methods
  ↳ cek data baru terima
- → Optimistic methods
  ↳ terima data baru cek

Exclusive lock, let only one tx access the affected object, ~~while~~ while concurring tx's that require access to the same object are rejected or queued.

Two-phase locking protocol (2PL) prevents a transaction from requesting an additional LOCK after the ~~first~~ first UNLOCK

Optimistic Method Phases
- Read phase
- Validation phase
- Write phase

Recovery : The restoration of a correct database state after an error.
↳ written in a log file

Nuanced Consistency Settings
- W + R <= N
- W = N
- R = N
- W + R > N.

Replication means keeping a copy of the same data on multiple machines that are connected via a network.
↳ algorithm :
- Single-leader : master-slave
- Multi-leader : e.g: multi-national DB
- Leaderless => e.g: offline access app.

Multi-leader topologies:
(a) Circular topology
(b) Star topology
(c) All-to-all topology

Partitions:
(a) Partition by key range
(b) Partition by hash of key